Who Stays On?

Keeping Shapla Shopping Complex alive through a day of load shedding

Who Stays On?

Keeping Shapla Shopping Complex alive through a day of load shedding

The story

It is 9:40 in the morning and Rokeya Apa, secretary of the Shapla Shopping Complex market committee in Mirpur, is standing outside the generator room holding the DESCO load-shedding schedule for today: **12 noon to 4 PM, and 5 PM to 8 PM.** Seven hours without grid power on a Thursday.

Shapla has one generator. Monir bhai, who runs it, has told her twice: *"Apa, it's a 20-kilowatt machine. The shops in this building together want nearly forty. Put them all on and it trips. And we have seventeen litres of diesel. Seventeen. The tanker is not coming before Saturday."*

So for every outage hour, somebody has to decide **who stays on and who sits in the dark.**

The pharmacy's insulin fridge and the bank's ATM booth are on every hour, no argument. The other ten shops all want power all day, and each has a case: Jamal the tailor has Eid orders; the mobile-phone shop is empty until 4 PM and packed from 5 to 8; the photocopy shop does its whole day before 3; the restaurant needs lunch and dinner and draws more than anyone; the electronics showroom sells after 5; the beauty salon has a wedding party booked 1 to 4; the grocery just wants its fridge; the jewellery, clothing and shoe shops want the evening crowd; Kashem's tea stall draws one kettle and one fan and wants to be left alone.

Two things make it hard. The generator burns diesel per **hour**, by how heavily it is loaded - not per shop. And **no shop may be left dark three hours in a row**; two hours people grumble, three and the owner is in her office with his brother-in-law.

Last year she did this with a pencil. The diesel ran out at 6:30 PM and the first floor went black in the evening rush. This year she has asked you.

What you are building

A **web app** that reads today's situation and produces the roster: for each outage hour, which shops are powered and which are dark, how much diesel is used, and why each dark shop is dark.

The screen shows, at minimum:

- the **roster board** - one row per shop with its name, one column per hour `10:00 … 19:00`, a ✓ or ✗ in every outage cell, grid-on hours shown as neutral "grid", mandatory shops marked;
- under the board, per hour: **combined load vs capacity**, and **fuel used that hour with the running total vs budget**;
- a **reason** for every ✗ (hover, click or a list);
- status, total value, total fuel, each shop's dark hours;
- load an input JSON (upload or paste); download the output JSON;

- a **print view**: the board, loads and fuel on one A4 page, for the generator-room door.

The API. Rokeya Apa has had calls from committees in Uttara and Chattogram who want the same tool. So the app must expose `POST /api/roster` - body = input JSON, response = output JSON - for any market to call. HTTP 200 for `ok` and `infeasible`, HTTP 400 for `invalid_input`. Under 10 seconds per input, no internet needed. Your own screen must use this same endpoint.

Input - one JSON file:

Field	Meaning
`grid_available`	10 booleans, hours 10:00 –20:00. `true` = grid on (everyone powered, nothing to decide). `false` = outage.
`generator_capacity_kw`	Most the generator can carry in any hour.
`fuel_budget_litres`	Diesel for the day.
`fuel_table`	Litres per hour at each load level. Look up the **combined** load; round up to the next level.
`max_consecutive_dark`	The longest a shop may be dark in a row (normally 2).
`shops[]`	`id`, `kw`, `type` (`mandatory` / `normal`), `value_per_slot` - 10 numbers: how much it is worth to have the shop on in each hour.

Output: the JSON your API returns: `status`; `roster` (for each outage hour, `powered` and `dark` ids, each dark entry with a one-line `reason`); `value_total`; `fuel_total_litres`; per-shop `dark_hours`. Exact formats: `schema.md` in your pack.

Your roster must never exceed capacity in any hour, never exceed the diesel budget, keep every mandatory shop on in every outage hour, never leave a normal shop dark more than `max_consecutive_dark` hours in a row, and - within those rules - serve as much value as possible.

You get: this brief, a small worked example (input, correct output and the arithmetic) that shows the formats, a twelve-shop practice input with no answer, a validator that checks any output against the rules, and the exact formats. You have 3.5 hours. At minute 150 everyone receives one updated input (DESCO extended the afternoon block). Re-run your app on it.

What makes a strong entry? Powering the most valuable shops hour by hour is the obvious start, and it fails: it spends the diesel at noon and leaves the evening dark, and it starves the same low-value shop until the three-hours rule breaks. Strong tools treat the day as one plan, compute fuel from the combined load per hour, and can explain every dark hour with the real rule or trade-off. A correct simple method is enough; a clear screen is required, decoration is not.

A worked example

A small market - five shops, a 10 kW generator, 8 litres - so the formats are clear. (Your practice input is the full twelve-shop one; this is just to read.)

```
{
 "grid_available": [true, true, false, false, false, true, true, false, false, false],
 "generator_capacity_kw": 10,
 "fuel_budget_litres": 8.00,
 "fuel_table": [ {"load_kw": 4, "litres_per_hour": 0.80},
                 {"load_kw": 7, "litres_per_hour": 1.20},
                 {"load_kw": 10, "litres_per_hour": 1.60} ],
 "max_consecutive_dark": 2,
 "shops": [
   {"id": "pharmacy",  "kw": 3, "type": "mandatory", "value_per_slot": [5,5,5,5,5,5,5,5,5,5]},
   {"id": "tailor",    "kw": 4, "type": "normal",    "value_per_slot": [2,2,3,3,4,4,4,4,5,5]},
   {"id": "mobile",    "kw": 3, "type": "normal",    "value_per_slot": [1,1,2,2,2,3,4,6,8,8]},
   {"id": "photocopy", "kw": 2, "type": "normal",    "value_per_slot": [3,3,5,5,4,3,2,2,1,1]},
   {"id": "tea_stall", "kw": 1, "type": "normal",    "value_per_slot": [2,2,2,3,3,2,2,3,3,3]}
 ]
}
```

How to read it:

- `grid_available` - ten entries, one per hour from 10:00. Here the grid is **off from 12:00 to 15:00 and from 17:00 to 20:00** (the `false` entries): six outage hours. Everything else is grid time, where nothing is decided.
- `generator_capacity_kw: 10` - in any outage hour, the kW of the powered shops added together must be ≤ 10.
- `fuel_table` - litres per hour by **combined** load: up to 4 kW costs 0.80 L, up to 7 kW 1.20 L, up to 10 kW 1.60 L. Pharmacy + tea stall = 4 kW → 0.80 L. Pharmacy + tailor + photocopy + tea stall = 10 kW → one lookup, 1.60 L (not four separate ones).
- `fuel_budget_litres: 8.00` - the day's diesel. The six outage hours together must stay within it.
- `max_consecutive_dark: 2` - a shop may be dark two hours in a row, never three. Grid hours reset the count.
- `shops` - the pharmacy (3 kW) is mandatory, so it is on in every outage hour and 7 kW remain for the other four. Each `value_per_slot` has ten numbers, one per hour: the mobile shop is worth 1 at 10:00 and 8 at 18:00; the photocopy shop is the reverse.

The best roster for this input scores **84** and uses exactly **8.00 L**:

shop	10:00	11:00	12:00	13:00	14:00	15:00	16:00	17:00	18:00	19:00
pharmacy ★	grid	grid	✓	✓	✓	grid	grid	✓	✓	✓
tailor	grid	grid	✗	✓	✗	grid	grid	✓	✗	✗
mobile	grid	grid	✓	✗	✗	grid	grid	✗	✓	✓
photocopy	grid	grid	✓	✓	✗	grid	grid	✓	✗	✗
tea_stall	grid	grid	✓	✓	✓	grid	grid	✓	✓	✓

shop	10:00	11:00	12:00	13:00	14:00	15:00	16:00	17:00	18:00	19:00
load / fuel	-	-	9 kW / 1.60	10 / 1.60	4 / 0.80	-	-	10 / 1.60	7 / 1.20	7 / 1.20

Read it: the tea stall (1 kW) is never cut, it is nearly free. The mobile shop gets 18:00 and 19:00, its best hours. 14:00 runs almost empty to save diesel for the evening. No row has three ✗ in a row. The output JSON for this roster is `example_output.json` in your pack.

The rules

1. **Hours.** Slots 0–9 are 10:00–20:00. A grid-on hour: everyone powered, no fuel, no roster entry, no value counted. A day with no outages is valid: empty roster, zero totals.
2. **Shops.** At most twelve. Unique non-blank `id`, positive whole-number `kw`, type `mandatory` or `normal`, ten whole-number values ≥ 0. An empty shop list is valid.
3. **Capacity.** In every outage hour, the kW of all powered shops added together ≤ capacity. A shop bigger than the generator can never be on during an outage (it runs on the grid like everyone else).
4. **Fuel.** The table lists load levels (whole kW, increasing, last one = capacity) with litres per hour. Add up the powered kW in the hour, find the first level that is at least that much, charge its litres - **once per hour on the combined load, never per shop**. Zero load = zero fuel. The day's total must be ≤ the budget. Work in whole hundredths of a litre; no rounding tolerance.
5. **Mandatory.** Every mandatory shop is on every outage hour. If no roster can satisfy all the rules together, answer `status: infeasible` and say which rule and which shops/hours are in conflict. A labelled best-effort roster may be attached.
6. **Dark runs.** For each normal shop, walk the ten hours in order: a dark outage hour adds one to its run; an on hour or a grid hour resets it to zero. No run may exceed `max_consecutive_dark`. The day starts with a run of zero. `dark_hours` counts all dark outage hours, not the longest run. Mandatory shops have zero.
7. **Objective.** Value = sum, over outage hours, of `value_per_slot[hour]` for every powered shop (mandatory included). Maximize it under rules 3–6. Equal-value rosters are equally good.
8. **Output.** `status` is `ok`, `infeasible` or `invalid_input`. For `ok`: every outage hour exactly once; `powered` and `dark` together are exactly the shop ids, no overlap, no repeats; every dark entry has a real reason (the rule or trade-off, with numbers); totals recomputed from the roster. `invalid_input` returns a message naming the field and exits non-zero (HTTP 400).
9. **Invalid input.** Anything that does not match the format in `schema.md` - a missing or mistyped field, wrong list lengths, duplicate ids, a bad fuel table, negative numbers, more than two decimals in litres - is `invalid_input`. A valid file whose rules cannot all be met is `infeasible`, not invalid.

10. **Minute-150 update.** One grid-on hour becomes an outage; same format; everything else unchanged; the same file for every team. Your old roster has no decision for the new hour - recompute.

Deliverables

One ZIP (or a repository link cloned before the freeze):

1. **The web app** - source plus one start command (`npm start`, `python app.py`, `docker compose up`, or `run.sh`) that serves the screen and `POST /api/roster` on a port stated in the README. No internet at runtime.
2. **README** - how to start it, where the solver lives, which files the API and screen call. Someone new to the code should find the decision logic in under two minutes.
3. **`evidence/`** - one input you built that broke an earlier version of your app, that version's output and the fixed output; one boundary-check input with its output. These must run against your API.
4. **Your outputs on the two public inputs** (the sample and the minute-150 update), committed in the repo.

Not required: a deployed URL, a demo video, a pitch, login, a database, mobile layout.

Rokeya Apa says thank you in advance. Monir bhai says don't trip his generator.